Residency Day 2 - Project 2: Code Injection via Web Apps and Dynamic Evaluation

Group 2

Bilal Khalid

Nischal Pokharel

Samia Zaman

Syed Kashif Ali

Steven Sisjayawan

Amer Mohammed

University of the Cumberlands

MSCS-535 Secure Software Development

2026 Spring

Prof. Brandon Bass

March 28, 2026

Introduction

This project demonstrates two critical web application vulnerabilities Cross-Site Scripting (XSS) and unsafe dynamic code evaluation via eval() along with their mitigations. Each vulnerability is implemented as a standalone Node.js web server for hands-on demonstration. Injection-based vulnerabilities have consistently ranked among the most critical threats to web application security, with XSS and code injection appearing repeatedly in the OWASP Top 10 list of risks (OWASP Foundation, 2021). Research has shown that the root cause of both vulnerability classes is the same: the absence of proper input validation before user-supplied data reaches a sensitive function (Faisal & Elshoush, 2023). This project illustrates how these attacks are carried out and how secure design principles can prevent them.

Cross-Site Scripting (XSS): Code Injection via Web Applications

What is XSS?

Cross-Site Scripting (XSS) is a vulnerability where an attacker injects malicious scripts into web pages viewed by other users. XSS has persisted as one of the most prevalent web application threats since its initial disclosure in the late 1990s, and despite over two decades of research, the attacks continue to rise in both frequency and sophistication (Hannousse & Yahiouche, 2024). In a reflected XSS attack, user input is included directly in the server's HTML response without sanitization. The browser cannot distinguish between the page's legitimate HTML and the attacker's injected code, so it executes everything.

Vulnerable Code

The server takes user input from a search form and inserts it directly into the HTML response:

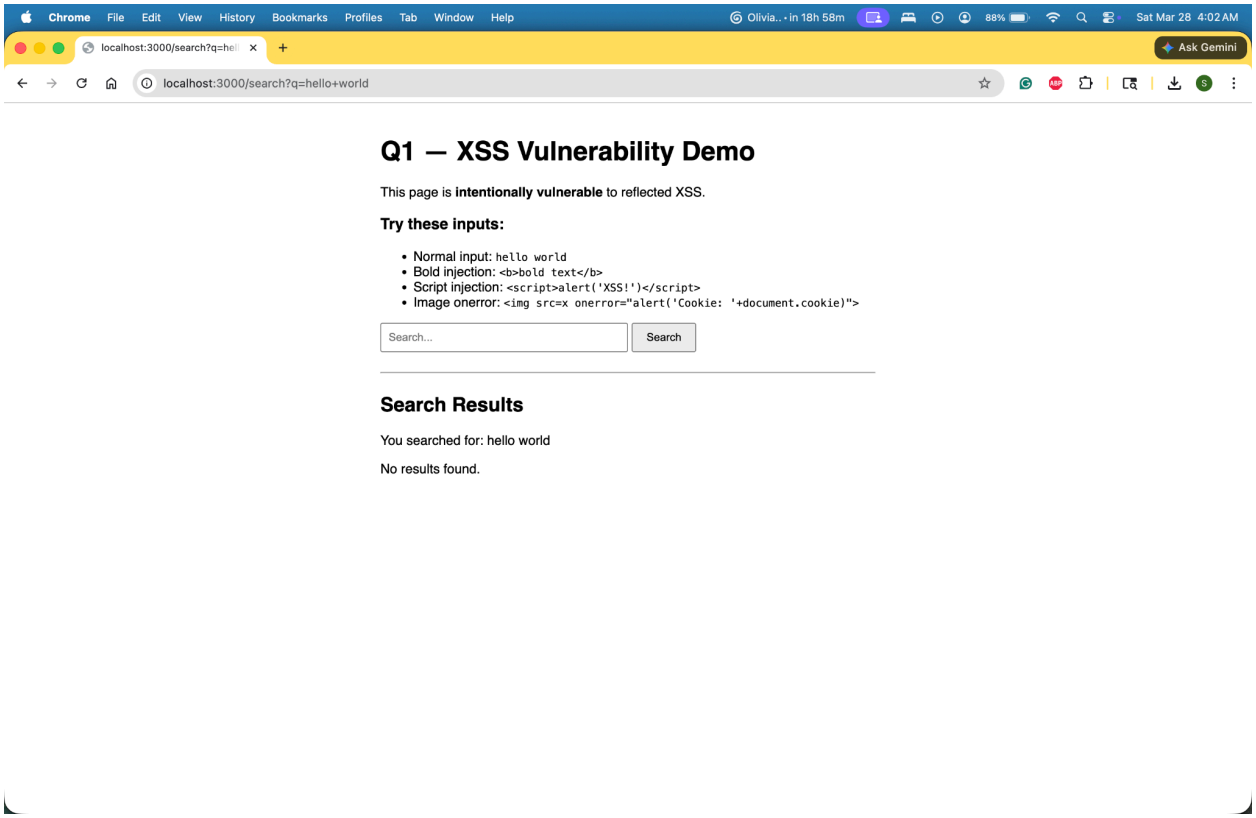
```
const query = parsed.query.q || "";

// VULNERABLE: user input is inserted directly into HTML
resultsHtml = `
  <h2>Search Results</h2>
  <p>You searched for: ${query}</p>
`;
```

The variable ${query} contains whatever the user typed, and it is placed into the HTML with no filtering or escaping.

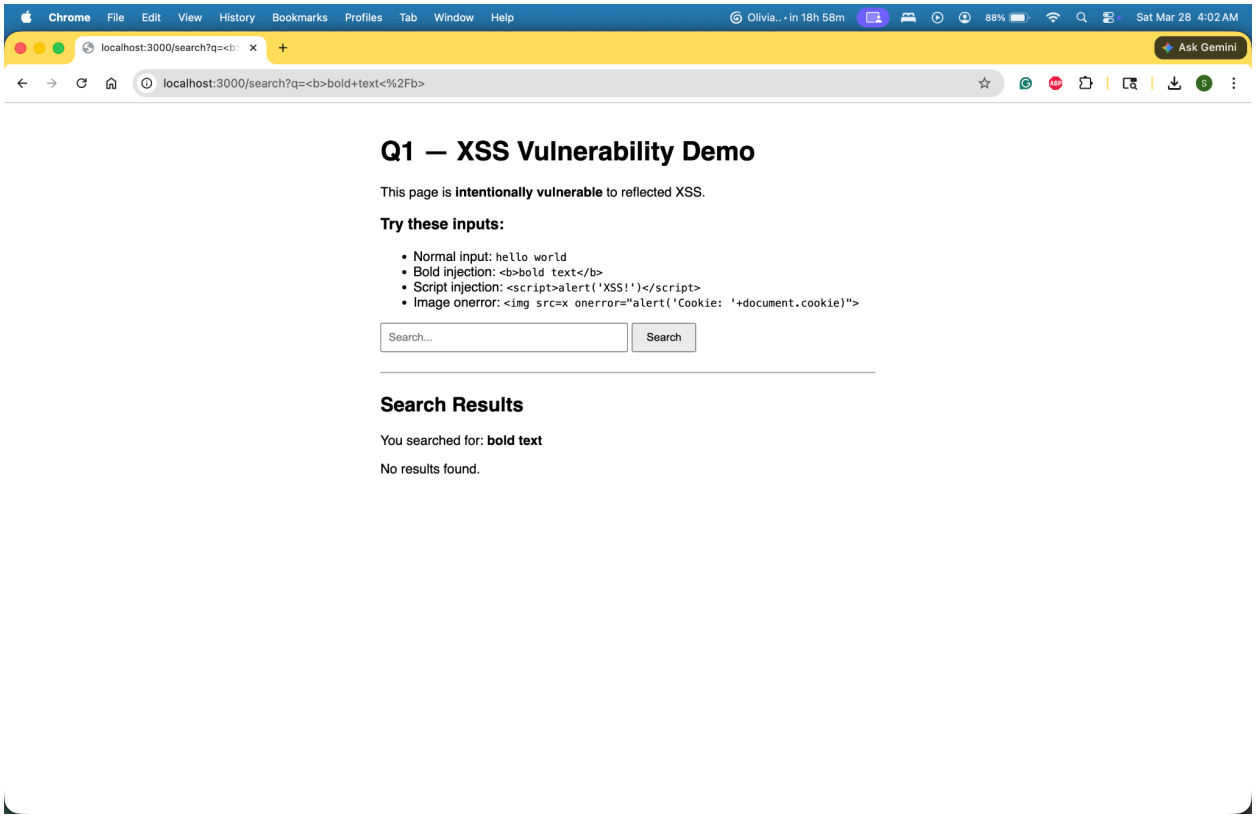
Demonstration

Normal input hello world: When a user searches for plain text like “hello world,” the page behaves as expected. The input is displayed back as text, and no malicious behavior occurs. This establishes a baseline showing the search feature works normally.



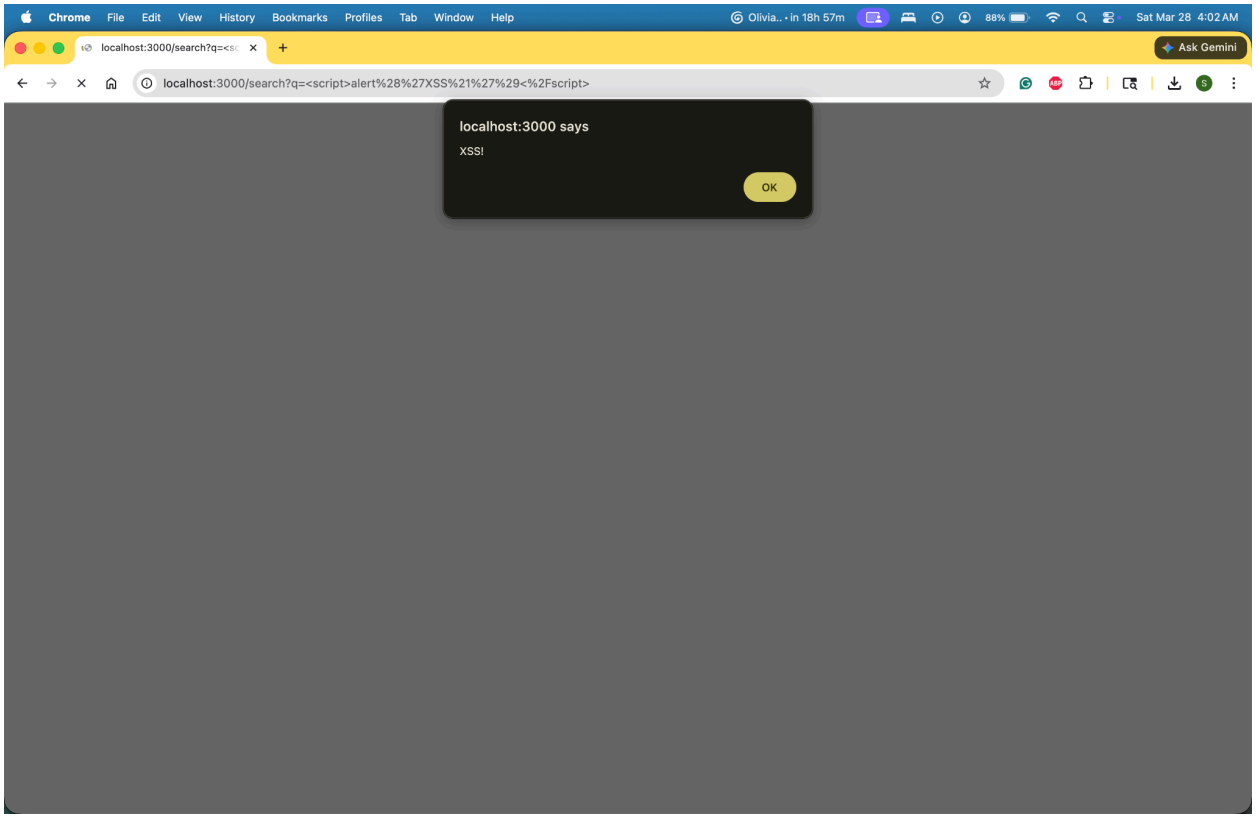
Screenshot 1. Search results page showing “You searched for: hello world”

HTML injection bold text: When a user enters HTML tags, the browser interprets them as real HTML. Instead of displaying the raw text, the page renders bold text. This proves the server is not distinguishing between user input and page markup.



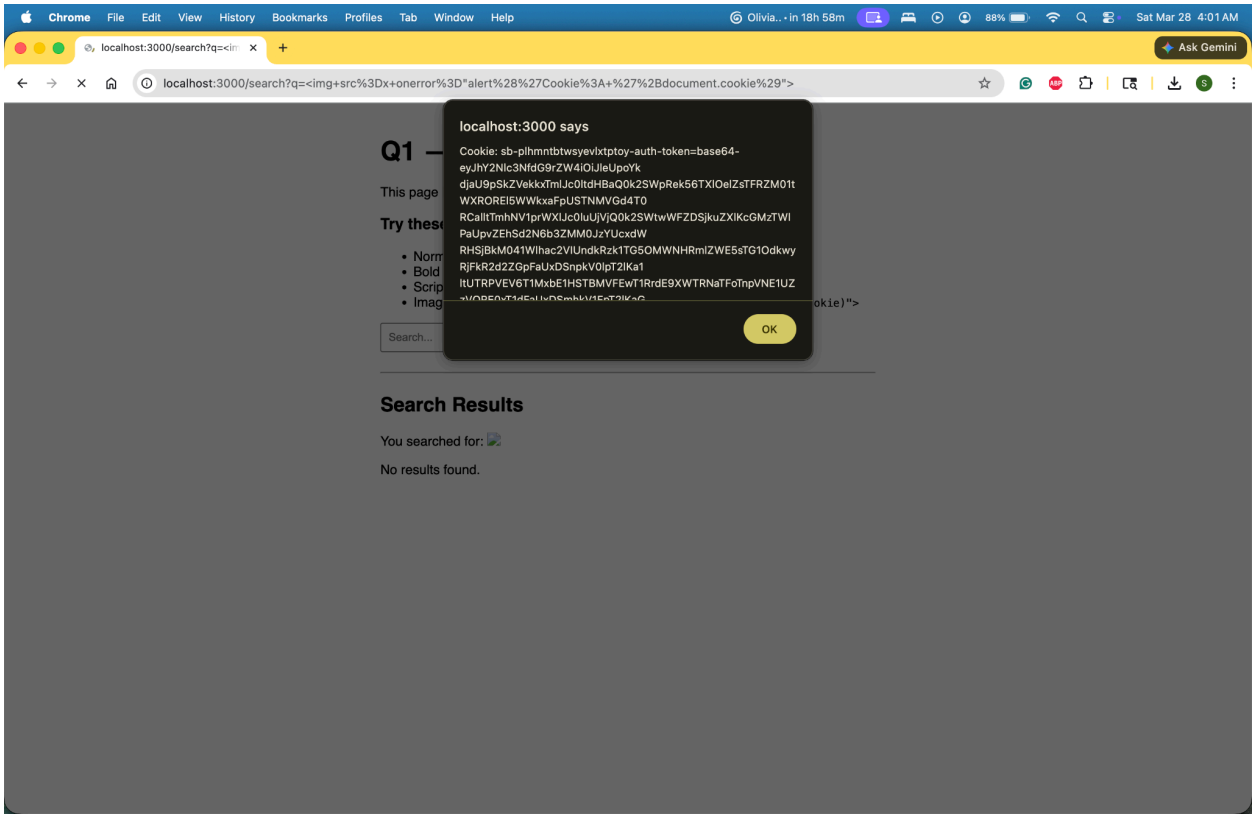
Screenshot 2. Search results page showing “bold text” rendered in bold

Script injection <script>alert(‘XSS!’)</script>: When a user enters a script tag, the browser executes the JavaScript. An alert dialog appears, proving that arbitrary code can run in the context of the page. In a real attack, this could steal cookies, redirect users, or modify page content. As Kaur et al. (2023) noted, XSS attacks can range in severity from minor annoyances to complete account compromise.



Screenshot 3. Browser alert dialog showing “XSS!”

Image onerror <img src=x onerror="alert(‘Cookie: ’+document.cookie)">: This payload uses an image tag with an invalid source. When the image fails to load, the onerror event fires and executes JavaScript. This demonstrates that XSS is not limited to <script> tags — many HTML elements can trigger code execution.



Screenshot 4. Browser alert dialog showing cookie information

Why This Is Dangerous

An attacker can craft a URL containing the malicious payload and send it to a victim. When the victim clicks the link, the script executes in their browser session. The attacker can steal session cookies, redirect to phishing pages, or modify what the user sees. The attack runs with the same permissions as the legitimate website.

Dynamic Evaluation of Code via eval()

What is Dynamic Code Evaluation?

Dynamic code evaluation means executing code that is constructed or received at runtime rather than code that was written at compile time. In JavaScript, eval() takes a string and runs it

as live code. When user input is passed to eval(), the attacker can execute arbitrary code on the server. The Common Weakness Enumeration classifies this under CWE-94: Improper Control of Generation of Code, noting that it allows attackers to craft input that modifies the intended control flow of the application (MITRE, 2024).

Vulnerable Code

The server takes a user-provided expression and passes it directly to eval():

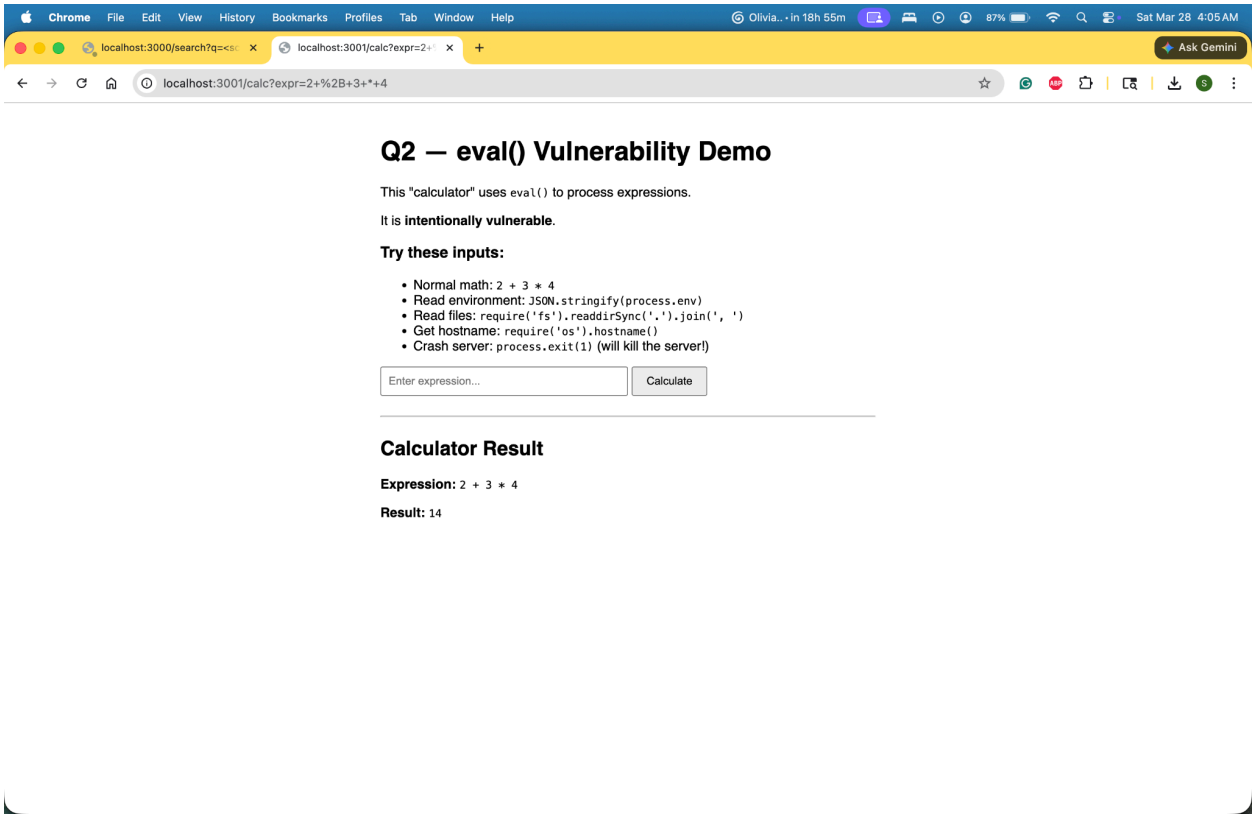
```
const expression = parsed.query.expr || "";

let result;
try {
  // VULNERABLE: eval() executes any JavaScript
  result = eval(expression);
} catch (e) {
  result = "Error: " + e.message;
}
```

There is no validation or restriction on what the expression can contain. Any valid JavaScript will execute on the server.

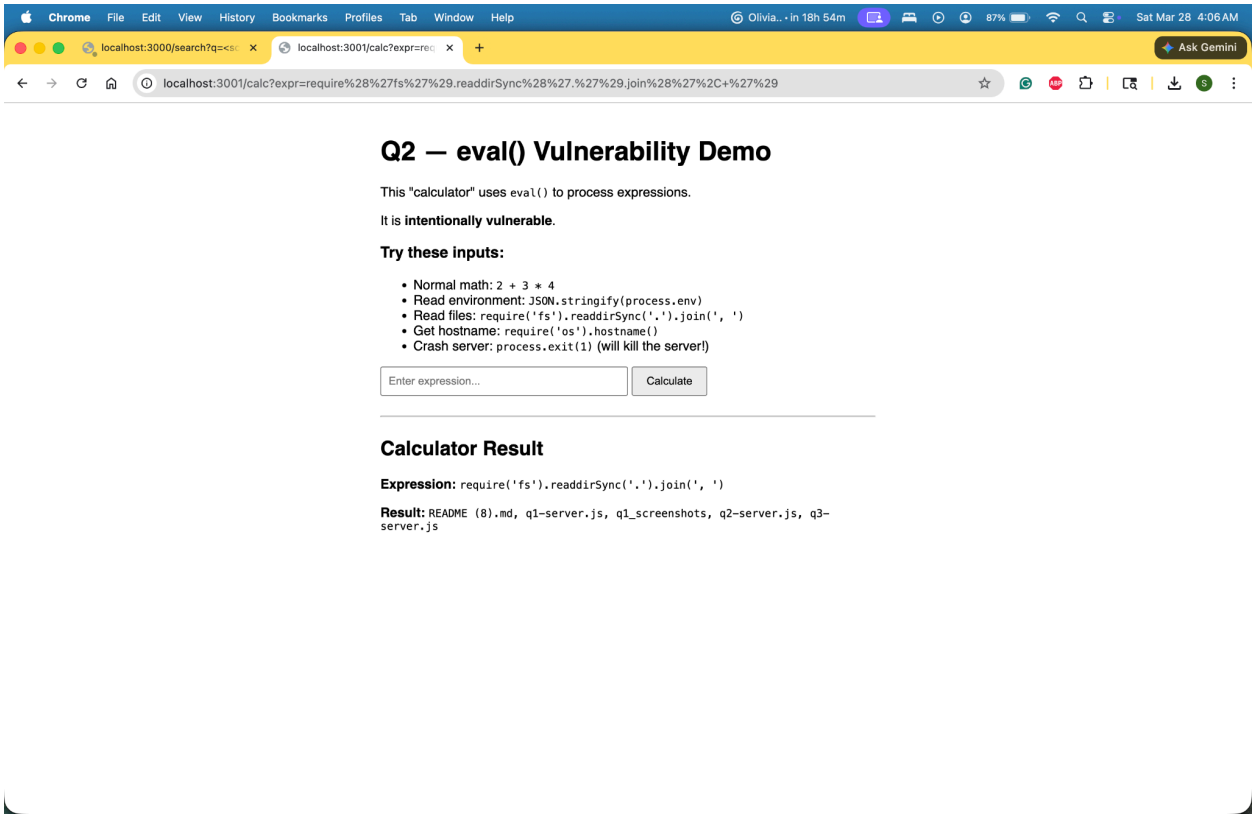
Demonstration

Normal math 2 + 3 * 4: When a user enters a math expression, eval() computes it and returns the result (14). This is the intended use case and demonstrates the calculator working normally.



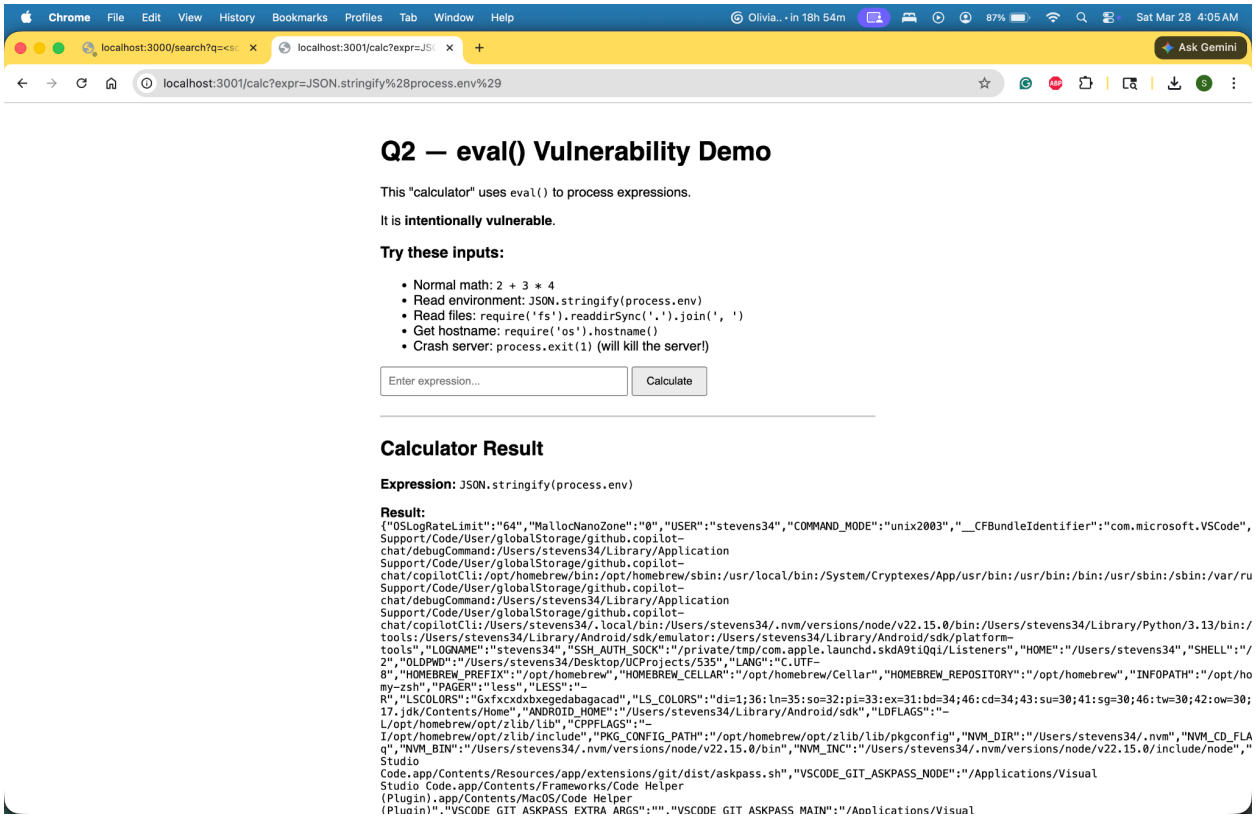
Screenshot 5. Calculator showing Expression: 2 + 3 * 4, Result: 14

Reading files require(‘fs’).readdirSync(‘.’).join(‘, ’): The attacker can read the server’s filesystem. This lists all files in the current directory, potentially exposing source code, configuration files, and credentials.



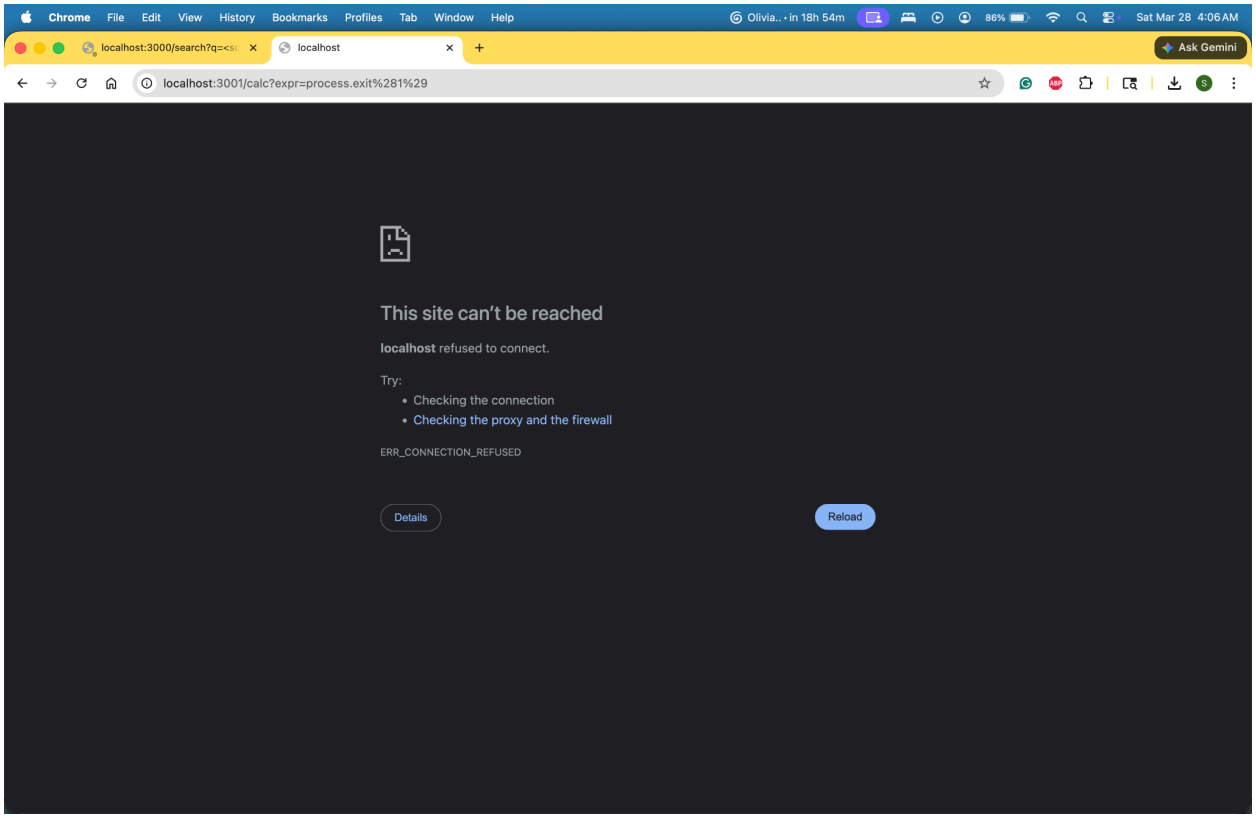
Screenshot 6. Calculator showing a list of files on the server

Reading environment variables JSON.stringify(process.env): Environment variables often contain database passwords, API keys, and other secrets. The attacker can dump all of them with a single expression.



Screenshot 7. Calculator showing environment variables

Crashing the server process.exit(1): The attacker can terminate the server process entirely, causing a denial of service for all users.



Screenshot 8. Server terminal showing the process has exited

Why This Is Dangerous

The attacker has full access to the Node.js runtime with the same privileges as the server process. They can read and write files, access databases, make network requests, and install backdoors. The OWASP Foundation (2021) identifies injection as one of the top three most critical web application security risks, emphasizing that any interface accepting user input and passing it to an interpreter without validation is susceptible to this class of attack.

Mitigations

XSS Mitigation: HTML Escaping

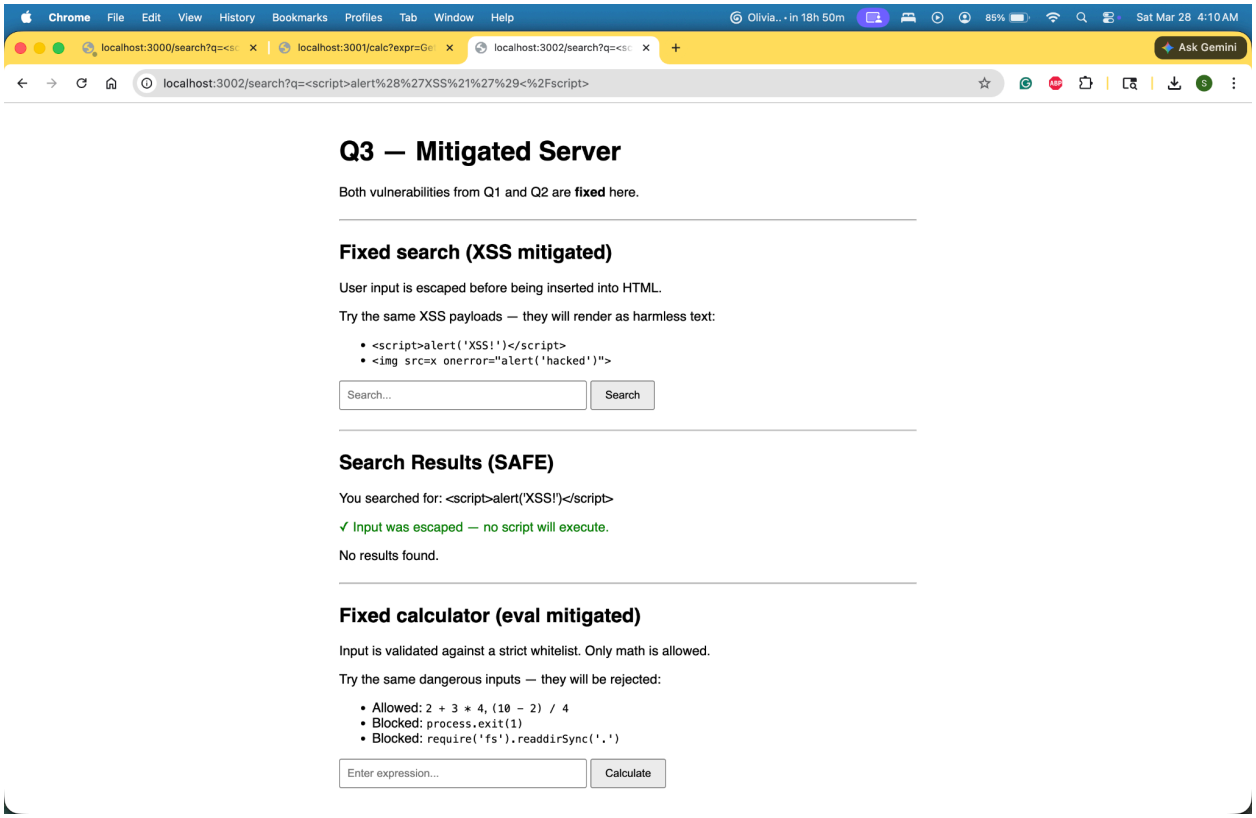
The fix converts dangerous characters to their HTML entity equivalents before inserting user input into the page. Output encoding is widely recognized as the primary defense against reflected and stored XSS attacks (Hannousse & Yahiouche, 2024):

```
function escapeHtml(str) {
  const map = {
    "&": "&amp;",  "<": "&lt;",
    ">": "&gt;",  '"': "&quot;",
    "'": "&#39;"
  };
  return str.replace(/[&<>"']/g, c => map[c]);
}

// Usage: input is escaped before being placed in HTML
const safeQuery = escapeHtml(query);
resultsHtml = `<p>You searched for: ${safeQuery}</p>`;
```

When a user types <script>alert(‘XSS!’)</script>, the escapeHtml() function converts it to <script>alert('XSS!')</script>. The browser renders this as visible text rather than executing it as code. The < and > characters are no longer treated as HTML tag delimiters. This ensures that the browser treats user-supplied content strictly as data to be displayed, never as executable instructions (Kaur et al., 2023).

When the same XSS payloads from Q1 are entered into the mitigated search form, they appear as plain text on the page. No scripts execute, no HTML renders the input is displayed exactly as the user typed it.



Screenshot 9. Mitigated search page showing script tag rendered as plain text with green checkmark

eval() Mitigation: Input Whitelist and Safe Evaluation

The fix uses a multi-layer defense:

```
function safeMathEval(expr) {
  // Layer 1: Whitelist — only math characters allowed
  if (!/^[\d\s+\-*/().%]+$/.test(expr)) {
    return { ok: false, error: "Rejected" };
  }

  // Layer 2: Function() for scope isolation
  const result = new Function("return (" + expr + ")")();

  // Layer 3: Validate result is a finite number
```

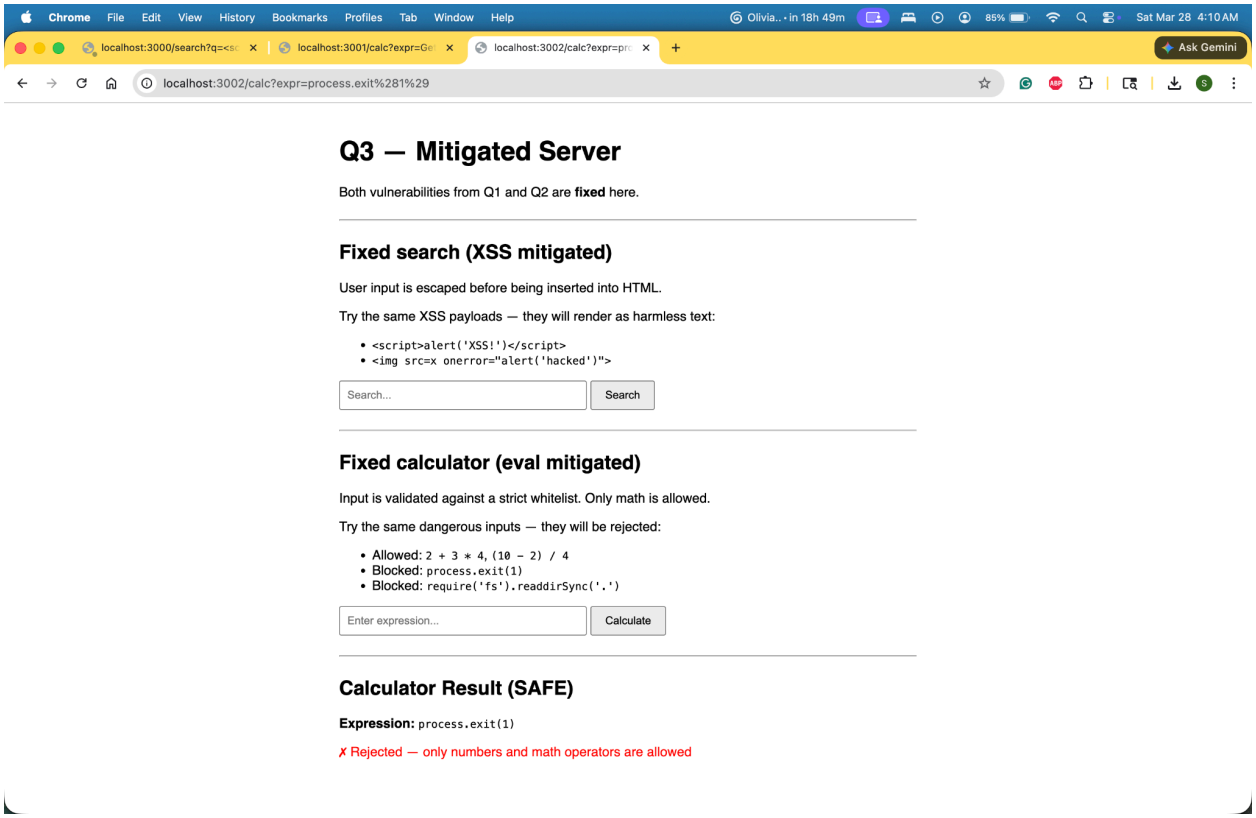
```
  if (typeof result !== "number" || !isFinite(result)) {
    return { ok: false, error: "Not a valid number" };
  }
  return { ok: true, result: result };
}
```

Input whitelist: A regular expression checks that the input contains only digits (0-9), math operators (+ - * / %), parentheses, decimal points, and spaces. Any input containing letters, semicolons, quotes, or other characters is immediately rejected. This blocks all dangerous inputs like require(...), process.exit(), and JSON.stringify(...) because they contain letters. As the MITRE CWE-94 advisory recommends, applications should avoid using functions like eval() entirely, and when dynamic evaluation is unavoidable, strict input validation against an allowlist of expected characters must be enforced (MITRE, 2024).

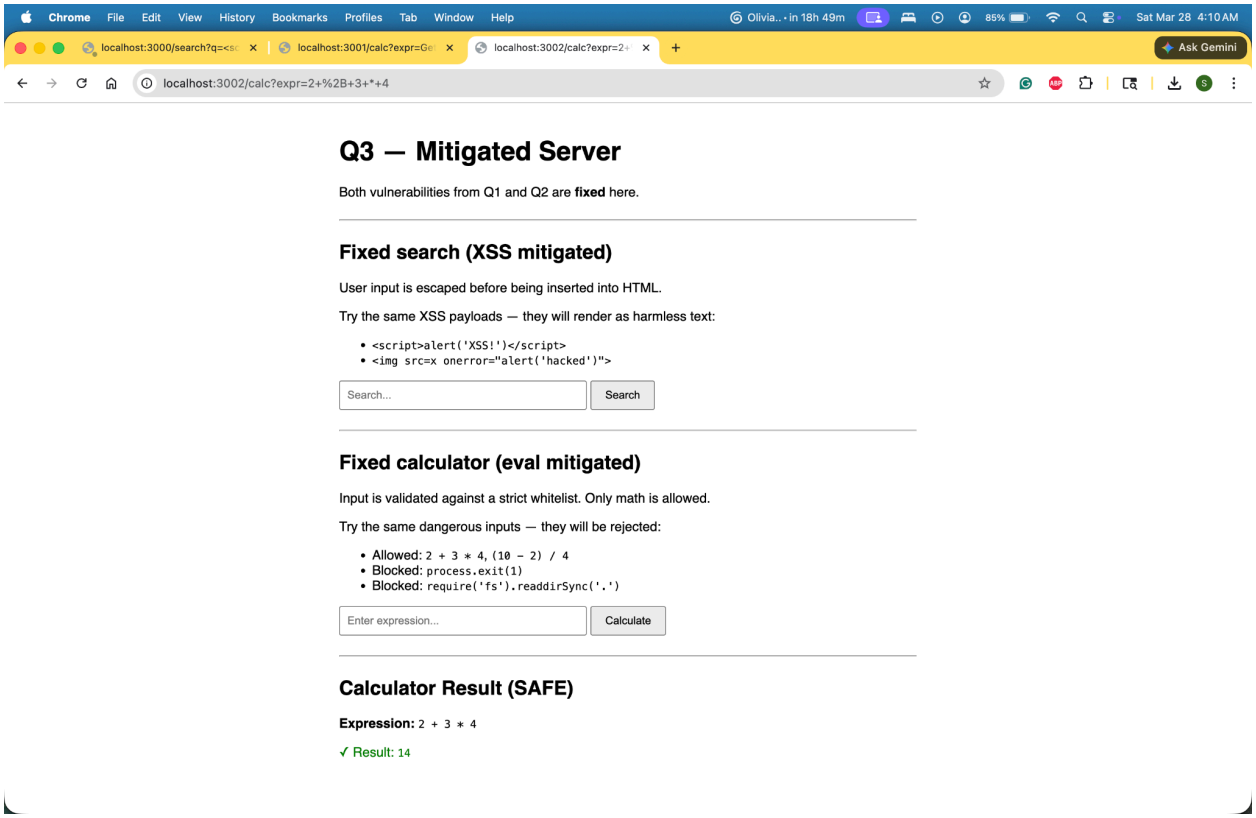
Scope isolation: Even after validation, new Function() is used instead of eval(). Unlike eval(), new Function() runs in its own scope and cannot access local variables.

Result validation: The output is verified to be a finite number, rejecting NaN or Infinity.

When tested, 2 + 3 * 4 still works correctly and returns 14 with a green checkmark. However, process.exit(1) is rejected with a red error stating “only numbers and math operators are allowed,” and require(‘fs’).readdirSync(‘.’) is similarly blocked.



Screenshot 10. Mitigated calculator rejecting process.exit(1) with red error message



Screenshot 11. Mitigated calculator successfully computing 2 + 3 * 4 with green checkmark

Conclusion

This project demonstrates how two common but critical web application vulnerabilities can be exploited and subsequently mitigated. The XSS example illustrates how an attacker can inject malicious scripts into a web page when user input is reflected in HTML without proper encoding. The eval() example shows how dynamic code evaluation gives an attacker full control over the server runtime when user input is passed to eval() without restriction. As Faisal and Elshoush (2023) concluded in their systematic review, the most effective protection against input validation vulnerabilities comes from combining multiple defense layers rather than relying on any single technique. The mitigated versions demonstrate that security measures do not need to sacrifice functionality the search and calculator features continue to work correctly while rejecting all malicious inputs.

GitHub Link: https://github.com/Samia1117/mscs-535-residency/tree/main/Project%202

References

Faisal, M., & Elshoush, H. (2023). Input validation vulnerabilities in web applications: Systematic review, classification, and analysis of the current state-of-the-art. *IEEE Access, 11*, 34216–34239. https://doi.org/10.1109/ACCESS.2023.3266385

Hannousse, A., & Yahiouche, S. (2024). Twenty-two years since revealing cross-site scripting attacks: A systematic mapping and a comprehensive survey. *Computer Science Review, 52*, 100621. https://doi.org/10.1016/j.cosrev.2024.100621

Kaur, J., Garg, U., & Bathla, G. (2023). Detection of cross-site scripting (XSS) attacks using machine learning techniques: A review. *Artificial Intelligence Review, 56*, 12725–12769. https://doi.org/10.1007/s10462-023-10433-3

MITRE. (2024). CWE-94: Improper control of generation of code (‘code injection’). *Common Weakness Enumeration*. https://cwe.mitre.org/data/definitions/94.html

OWASP Foundation. (2021). OWASP Top 10: The ten most critical web application security risks. https://owasp.org/www-project-top-ten/